Лекция 7. Производные типы данных

20 октября 2026 г.

1 MPI_Type_create_struct

Для создания и регистрации в MPI структуры используется функция MPI_Type_create_struct, которая имеет следующую сигнатуру:

```
MPI_Type_create_struct(
        int count,                              //in
        int array_of_blocklength[],             //in
        MPI_Aint array_of_sizes[],              //in
        MPI_Datatype array_of_types[],          //in
        MPI_Datatype* new_type                  //out
);
```

Функция принимает следующие аргументы:

1. Количество передаваемых элементов. Каждый элемент может быть либо одиночной переменной, либо массивом.

2. Массив длин, в котором указывается, сколько переменных в каждом элементе мы передаем.

3. Массив смещений адреса относительно первой передаваемой переменной.

4. Массив типов. Для каждого элемента нужно указать, какой у него тип.

5. Указатель на создаваемый тип, по которому он и будет записан.

Длина каждого из трех массивов должна составлять `count` элементов.

Следующая функция является «оберткой» для вызова `MPI_Type_create_struct`, которую мы используем для создания структуры из двух дробных чисел и одного целого:

```
1  void Build_mpi_type(double *a_p, double *b_p,
2        int *n_p, MPI_Datatype *mpi_derived_datatype){
3    int array_of_block_lengths[3] = {1, 1, 1};
4    MPI_Aint a_addr, b_addr, n_addr;
5    MPI_Get_address(a_p, &a_addr);
6    MPI_Get_address(b_p, &b_addr);
7    MPI_Get_address(n_p, &n_addr);
8    MPI_Aint array_of_displacements[3] = {0, b_addr - a_addr,
9        n_addr - a_addr};
10   MPI_Datatype array_of_types[3] = {MPI_DOUBLE,
11       MPI_DOUBLE, MPI_INT};
12   MPI_Type_create_struct(3, array_of_block_lengths,
13         array_of_displacements, array_of_types, mpi_derived_datatype);
14   MPI_Type_commit(mpi_derived_datatype);
15 }
```

Созданный тип регистрируется в `MPI` в 14 строке.

После использования типа (любого пользовательского типа, не обязательно структуры) нужно освободить выделенные для него ресурсы с помощью функции

```
MPI_Type_free(
  MPI_datatype* mpi_dt   /*in/out*/
);
```

Пример использования нового типа данных:

```
void Input(){
  MPI_Datatype abn;
  Build_mpi_type(&a, &b, &n, &abn);
  if (my_rank == 0)
    scanf("%lf %lf %d", &a, &b, &n);
  MPI_Bcast(&a, 1, abn, 0, MPI_COMM_WORLD);
  MPI_Type_free(&abn);
}
```

2 MPI_Type_contiguous

Создание нового типа данных, представляющего собой `count` идущих подряд элементов типа `oldtype`:

```
int MPI_Type_contiguous(
    int count,                  /*in*/
    MPI_Datatype oldtype,       /*in*/
    MPI_Datatype* newtype       /*out*/
);
```

Эту функцию удобно использовать в том случае, когда требуется прочитать строку или столбец матрицы целиком. Так, если определен тип `type_row`

```
MPI_Datatype type_row;
MPI_Type_contiguous(c, MPI_INT, &type_row);
MPI_Type_commit(&type_row);
```

то вызов `MPI_Scatter` из функции заполнения матрицы `InputMatrix` из предыдущего семинара можно переписать проще, а именно, вместо

```
MPI_Scatter(temp, local_r * c, MPI_INT, mat, local_r * c,
        MPI_INT, 0, MPI_COMM_WORLD);
```

написать

```
MPI_Scatter(temp, local_r, type_row, mat, local_r * c,
        MPI_INT, 0, MPI_COMM_WORLD);
```

3 MPI_Type_vector

Создание блочно-циклического разбиения вектора:

```
int MPI_Type_vector(
        int count,
        int blocklength,
        int stride,
        MPI_Datatype oldtype,
        MPI_Datatype *newtype
);
```

где `count` — количество блоков;
`blocklength` — длина блока;
`stride` — количество элементов между началом одного блока и началом другого;
`oldtype` — тип элементов блока;
`newtype` — указатель на создаваемый тип данных.

Продемонстрируем возможности функции `MPI_Type_vector` на примере блочко-циклического разбиения вектора. Длина вектора равна 18. Длина блока и количество блоков равны двум и трем соответственно.

```
1  #include <mpi.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  MPI_Datatype type_vec, type_vec_new;
6
7  int block_len = 2;
8  int block_num = 3;
9  int n = 18;
10
11 // 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
12 // block 1: 0 1 6  7  12 13
13 // block 2: 2 3 8  9  14 15
14 // block 3: 4 5 10 11 16 17
15
16 void InputVector(int* v, int n, int my_rank, int comm_sz){
17   int* temp = calloc(n,sizeof(int));
18   if(my_rank == 0){
19     for(int i=0; i<n; ++i)
20     temp[i] = i;
21   }
22   MPI_Scatter(temp,1,type_vec_new,v,n/comm_sz,MPI_INT,0,MPI_COMM_WORLD);
23   free(temp);
24 }
25 void PrintDistributedVector(int* v, int n, int my_rank, int comm_sz){
26   int* temp = calloc(n,sizeof(int));
27   MPI_Gather(v,n/comm_sz,MPI_INT,temp,1,type_vec_new,0,MPI_COMM_WORLD);
28   if(my_rank==0)
```

```
29    for(int i=0; i<n; ++i)
30    printf("%3d",temp[i]);
31    printf("\n");
32    free(temp);
33 }
34
35 int main(){
36
37    int my_rank;
38    int comm_sz;
39
40    MPI_Init(NULL, NULL);
41
42    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
43    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
44
45    MPI_Type_vector(block_num, block_len, n/comm_sz, MPI_INT, &type_vec);
46
47    MPI_Aint l,e;
48    MPI_Type_get_extent(type_vec,&l,&e);
49    MPI_Type_create_resized(type_vec,l,block_len*sizeof(int),&type_vec_new
50    MPI_Type_commit(&type_vec_new);
51
52    int* v = calloc(n,sizeof(int));
53    InputVector(v,n,my_rank,comm_sz);
54    PrintDistributedVector(v,n,my_rank,comm_sz);
55
56    free(v);
57    MPI_Type_free(&type_vec);
58    MPI_Type_free(&type_vec_new);
59    MPI_Finalize();
60
61 }
```

Отметим следующее важное обстоятельство. Тип, создаваемый `MPI_Type_vector`, нельзя использовать с функциями `MPI_Gather` и `MPI_Scatter`. Дело в том, что функции `MPI_Gather` и `MPI_Scatter` для разделения достающихся различным процессам «кусков» массива используют такой

параметр, как «ширина» (`extent`). Каждый пользовательский тип данных имеет ширину, значение которой по умолчанию — это расстояние (в байтах) между началом и концом фрагмента данных. В рассматриваемом примере ширина равна $14 \cdot \text{sizeof(int)} = 56$. Однако ясно, что для перехода от текущего блока к следующему нужно выполнить сдвиг на $2 \cdot \text{sizeof(int)} = 8$ байта. Следовательно, требуется переписать ширину созданного типа. Для этого нужно создать новый тип на базе существующего, но с другой шириной, что и делается в строке 49. Узнать текущее значение ширины типа можно с помощью функции MPI_Type_get_extent, которая вызывается в строке 48.

4 MPI_Type_indexed

Данная функция позволяет создать произвольный набор компонент массива.

```
int MPI_Type_indexed(
        int count,
        const int *array_of_blocklengths,
        const int *array_of_displacements,
        MPI_Datatype oldtype,
        MPI_Datatype *newtype
);
```

В качестве примера рассмотрим выделение правой верхней подматрицы из следующей матрицы

$$\begin{pmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 \end{pmatrix}$$

Предполагается, что матрица хранится в виде одномерного массива. Мы выделим из него четыре фрагмента с длинами 4, 3, 2 и 1. Смещения для фрагментов (индекс в массиве, от которого отсчитываем длину) будут 0, 5, 10 и 15.

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
```

```
int main(){

  int comm_sz;
  int my_rank;

  MPI_Init(NULL, NULL);

  comm_sz = 2;
  //MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
  MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
  MPI_Datatype type_indexed;
  int array_of_blocklenghts[4] = {4, 3, 2, 1};
  int array_of_displscements[4] = {0, 5, 10, 15};

  MPI_Type_indexed(4, array_of_blocklenghts, array_of_displscements,
    MPI_INT, &type_indexed);
  MPI_Type_commit(&type_indexed);
  int mat[16],mat1[10];
  if (my_rank == 0){
    for (int i = 0; i < 16; ++i){
      mat[i] = i;
    }
    MPI_Send(mat,1,type_indexed,1,0,MPI_COMM_WORLD);
  }else{
    MPI_Recv(mat1,10,MPI_INT,0,0,
    MPI_COMM_WORLD,MPI_STATUS_IGNORE);
    for (int i=0;i<10;++i){
      printf("%3d",mat1[i]);
    }
    printf("\n");
  }

  MPI_Type_free(&type_indexed);
  MPI_Finalize();
}
```

5 MPI_Pack и MPI_Unpack

Альтернативой созданию пользовательских типов данных является практика запаковки и распаковки данных.

```
int MPI_Pack(
        const void *inbuf,
        int incount,
        MPI_Datatype datatype,
        void *outbuf,  // out
        int outsize,
        int *position, // in/out
        MPI_Comm comm
);

int MPI_Unpack(
        const void *inbuf,
        int insize,
        int *position, // in/out
        void *outbuf,  // out
        int outcount,
        MPI_Datatype datatype,
        MPI_Comm comm
);
```

Листинг 1: Демонстрация запаковки и распаковки данных

```
void InputDim(double* a, double* b, int *n, int my_rank)
{
   char pack_buf[100];
   int position=0;
   if (my_rank == 0)
   {
      scanf("%d", n);
      scanf("%lf",a);
      scanf("%lf",b);
      MPI_Pack(a,1,MPI_DOUBLE,pack_buf,100,&position,MPI_COMM_WORLD);
      MPI_Pack(b,1,MPI_DOUBLE,pack_buf,100,&position,MPI_COMM_WORLD);
      MPI_Pack(n,1,MPI_INT,pack_buf,100,&position,MPI_COMM_WORLD);
   }
```

```
    MPI_Bcast(pack_buf, 100, MPI_PACKED, 0, MPI_COMM_WORLD);
    position=0;
    MPI_Unpack(pack_buf,100,&position,a,1, MPI_DOUBLE, MPI_COMM_WORLD);
    MPI_Unpack(pack_buf,100,&position,b,1, MPI_DOUBLE, MPI_COMM_WORLD);
    MPI_Unpack(pack_buf,100,&position,n,1, MPI_INT, MPI_COMM_WORLD);
    if(my_rank==1){
        printf("%lf %lf %d\n", *a, *b, *n);
    }
}
```